Introduction to Computing (CS 101)

Conditional Statements and Loops

T. Venkatesh & John Jose

Department of Computer Science & Engineering

Indian Institute of Technology Guwahati

Type Casting

- Some languages are strict on the type of data used for an operation, addition of two different type of object
  - You can not add **a mango** and **an apple**
  - Operating on two data types may result in errors
- Implicit type casting - auto upgradation and demotion
- Explicit type casting

```
int I;  float  F;
I=F;    //auto demotion
F=I;   //auto promotion
I=(int) F;  //manual demotion
F=(float) I; //manual promotion
```

Type Casting Floating Point

- Casting between `int`, `float`, and `double` changes numeric values
- `double` or `float` to `int`
  - Truncates fractional part
  - Like rounding toward zero
  - Not defined when out of range
- `int` to `float`
  - Exact conversion, as long as int has ≤ 23 bit mantissa
  - Will round according to rounding mode
- `int` to `double`
  - Exact conversion, as long as int has ≤ 53 mantissa

Answers to Floating Point Puzzles

```
int x = …;

float f = …;

double d = …;
```

Assume neither `d` nor `f` is NAN

- `x==(float)x;` Comparison Result?
- `x==(int)(double)x;` Comparison Result?
- `f==(float)(double)f;` Comparison Result?
- `d==(float)d;` Comparison Result?
- `f==-(-f);` Comparison Result?

Answers to Floating Point Puzzles

```
int x = …;

float f = …;

double d = …;
```

Assume neither `d` nor `f` is NAN

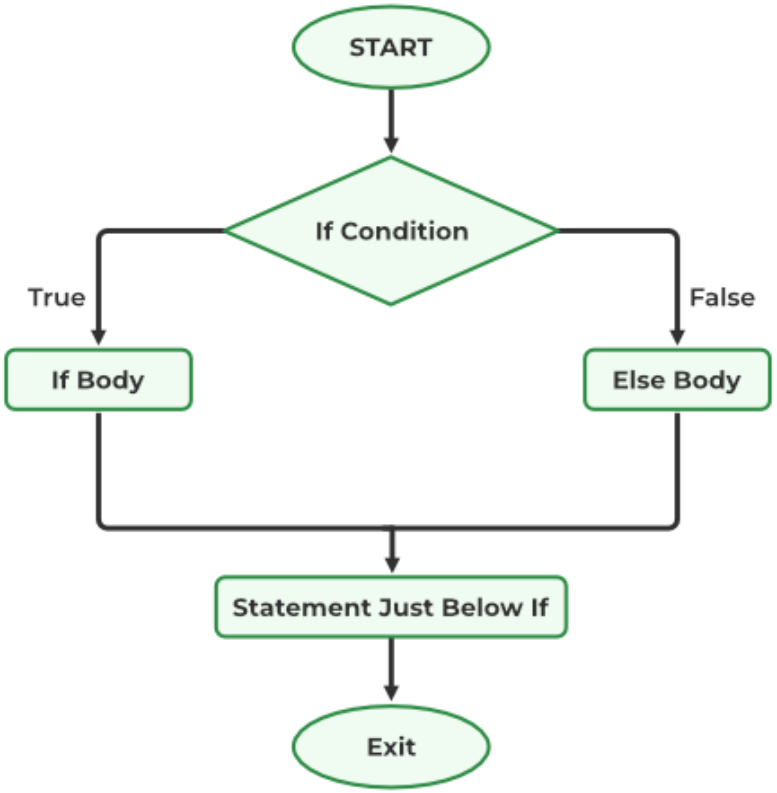
- `x==(float)x;` — No: will not match
- `x==(int)(double)x;` — Yes: 53 bit Fraction
- `f==(float)(double)f;` — Yes: increases precision
- `d==(float)d;` — No: loses precision
- `f==-(-f);` — Yes: Just change sign bit

Structured Programming

- All programs can be written in terms of only three control structures
  - Sequence, selection and repetition
- **The sequence structure**
  - Unless otherwise directed, the statements are executed in the order in which they are written.
- **The selection structure**
  - Used to choose among alternative courses of action.
- **The repetition structure**
  - Allows an action to be repeated while some condition remains true.

Selection: the if-else statement

```
if  (condition )
{
  statement(s)
}
else
{
      statement(s)
}
```



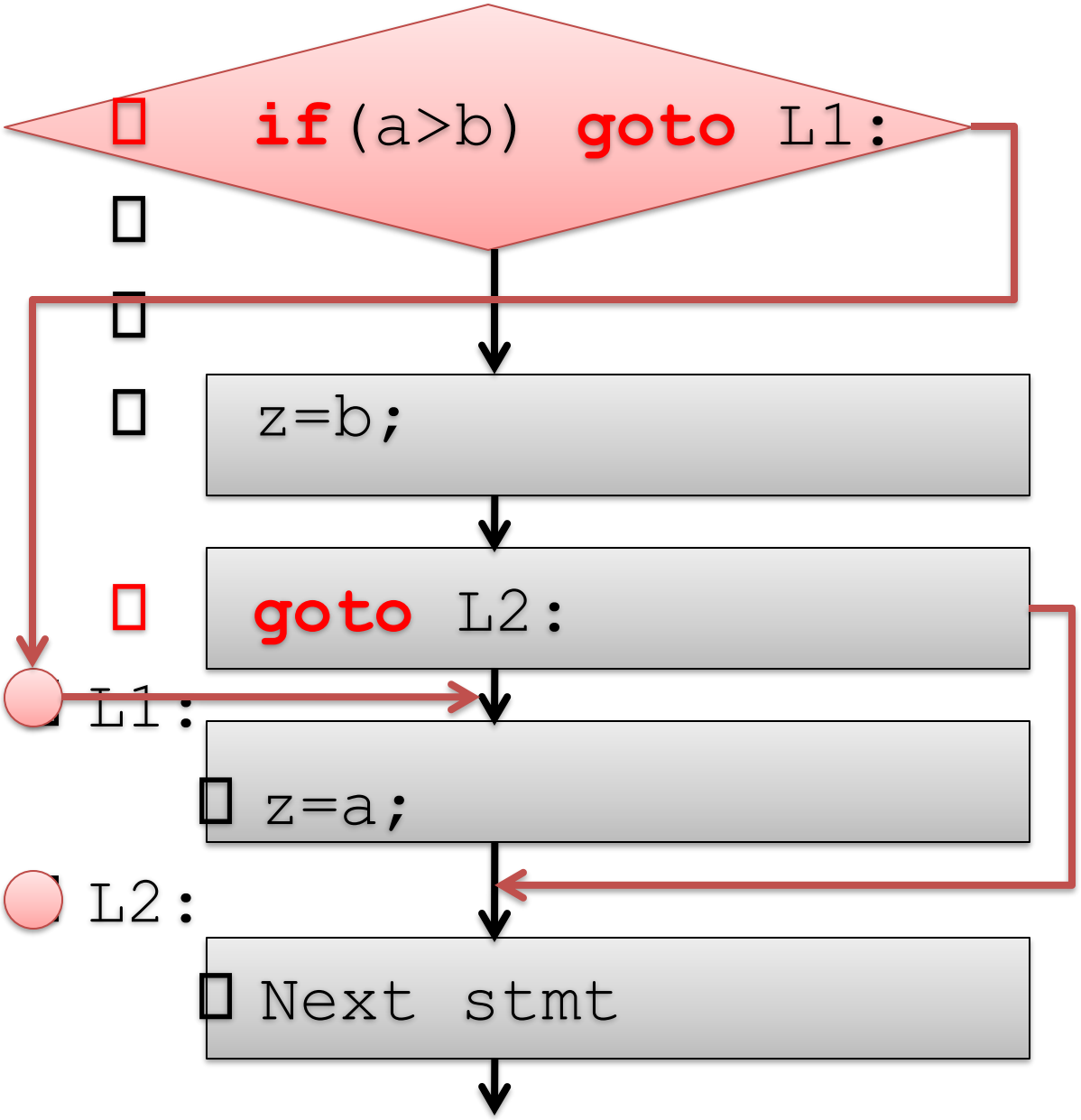
In Assembly language: No if-else

- In Assembly language no support for if else
- They get implemented using **if** and **goto** statement
- **goto** statement uses **Label**
- **If else** get converted to **if goto**

```
if(a> b )
      z=a;
else z=b;
NextStmt;
```

```
    if(a>b) goto L1:
    z=b;
    goto L2:
L1: z=a;
L2: NextStmt;
```

In Assembly language: No if-else



Multi-way if else : switch case

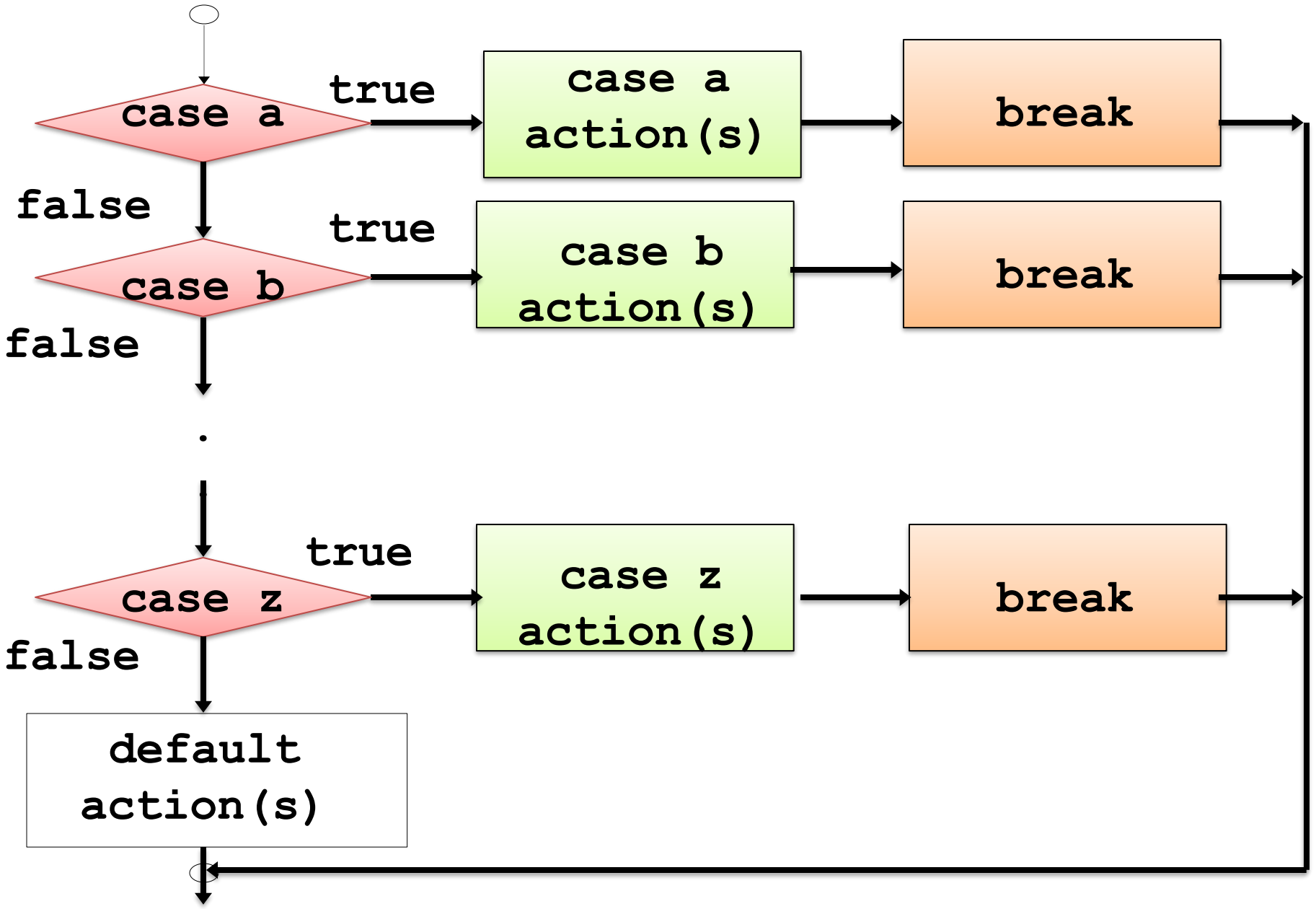
- If-else : two way, if part and else part
- To make it multi-way: nested if-else
  - Complex and lengthy
- C language provide
  - Switch case
  - Multi way selection

The switch Multiple-Selection Structure

- **switch**
  - Useful when expression is tested for multiple values
  - Consists of a series of **case** labels and an optional **default** case
  - **break** is (almost always) necessary

```
switch (<expression>){
  case <Value1> :
     <Action/Stmts for Value1>; break;
  case <Value2> :
     <Action/Stmts for Value2>; break;
  . . .
  default: <Action/Stmts for DefaultValue>;
           break;
}
```

Flowchart of Switch Statement



Multiway Switch Selection example

```
int main(){//simple calculator
 int a=50,b=10, R;
 char choice;
 printf("Enter choice");
 scanf("%c",&choice);

 switch (choice){
  case 'a' : R=a+b; printf("R=%d",R); break;
  case 's' : R=a-b; printf("R=%d",R); break;
  case 'm' : R=a*b; printf("R=%d",R); break;
  case 'd' : R=a/b; printf("R=%d",R); break;
  default  : printf("Wrong choice") ; break;
   }
 return 0;
}
```

Multiway Switch Selection example

```
int main(){//simple calculator
 int a=50,b=10, R;
 char choice;
 printf("Enter choice");
 scanf("%c",&choice);

 switch (choice){
  case 'a' : R=a+b; printf("R=%d",R); break;
  case 's' : R=a-b; printf("R=%d",R); break;
  case 'm' : R=a*b; printf("R=%d",R); break;
  case 'd' : R=a/b; printf("R=%d",R); break;
  default  : printf("Wrong choice") ; break;
   }
 return 0;
}
```

Loops and Repetition

- Loops in programs allow us to repeat blocks of code
- Useful for:
  - Counting
  - Repetitive activities
  - Programs that never end
- Three Types of Loops/Repetition in C
- **while**
  - top-tested loop (pretest)
- **for**
  - counting loop
  - forever-sentinel
- **do**
  - bottom-tested loop (posttest)

The while loop

- Top-tested loop (pre-test) or entry controlled loop

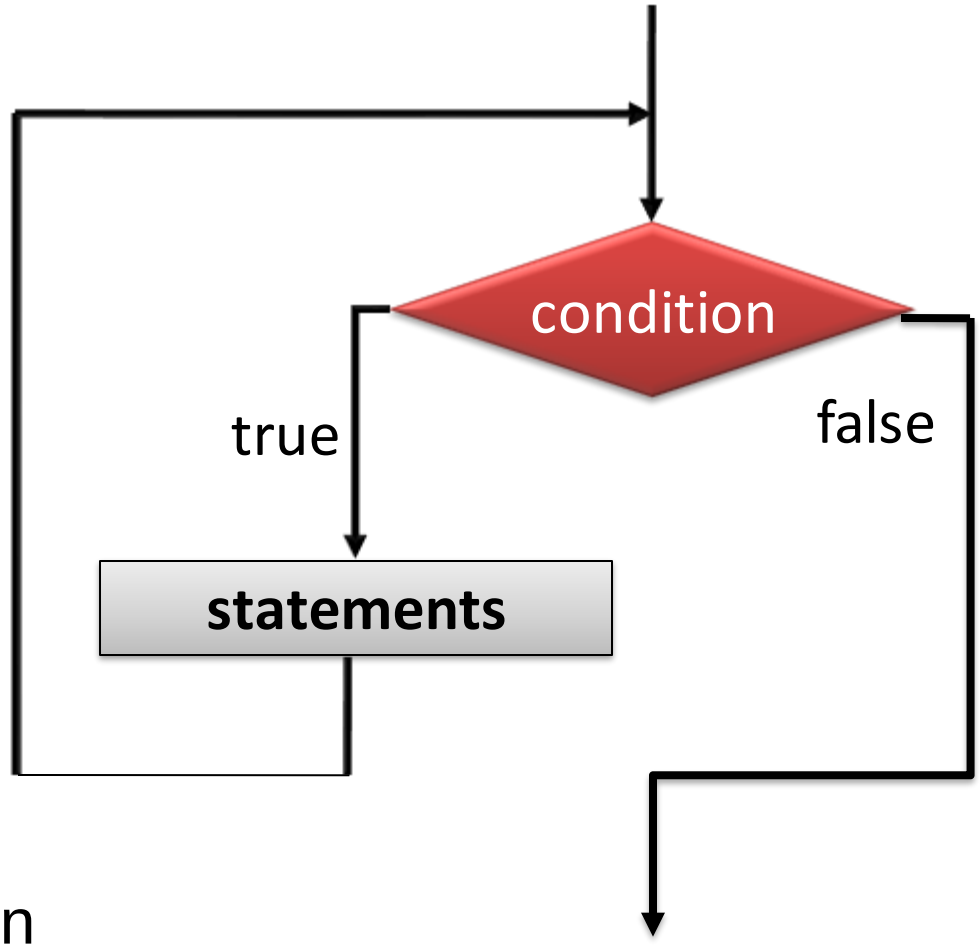
```
while (condition)
   statement;
```

- Like if, only one statement is executed as while body
- For a block to repeat more than one statement (using { })

```
while (condition){
   statements;
}
```

while(condition)statement;

```
while (condition){
   statements;
}
```



- Check the **Boolean** condition
- If true, execute the statement/block
- Repeat the above until the **Boolean** is false

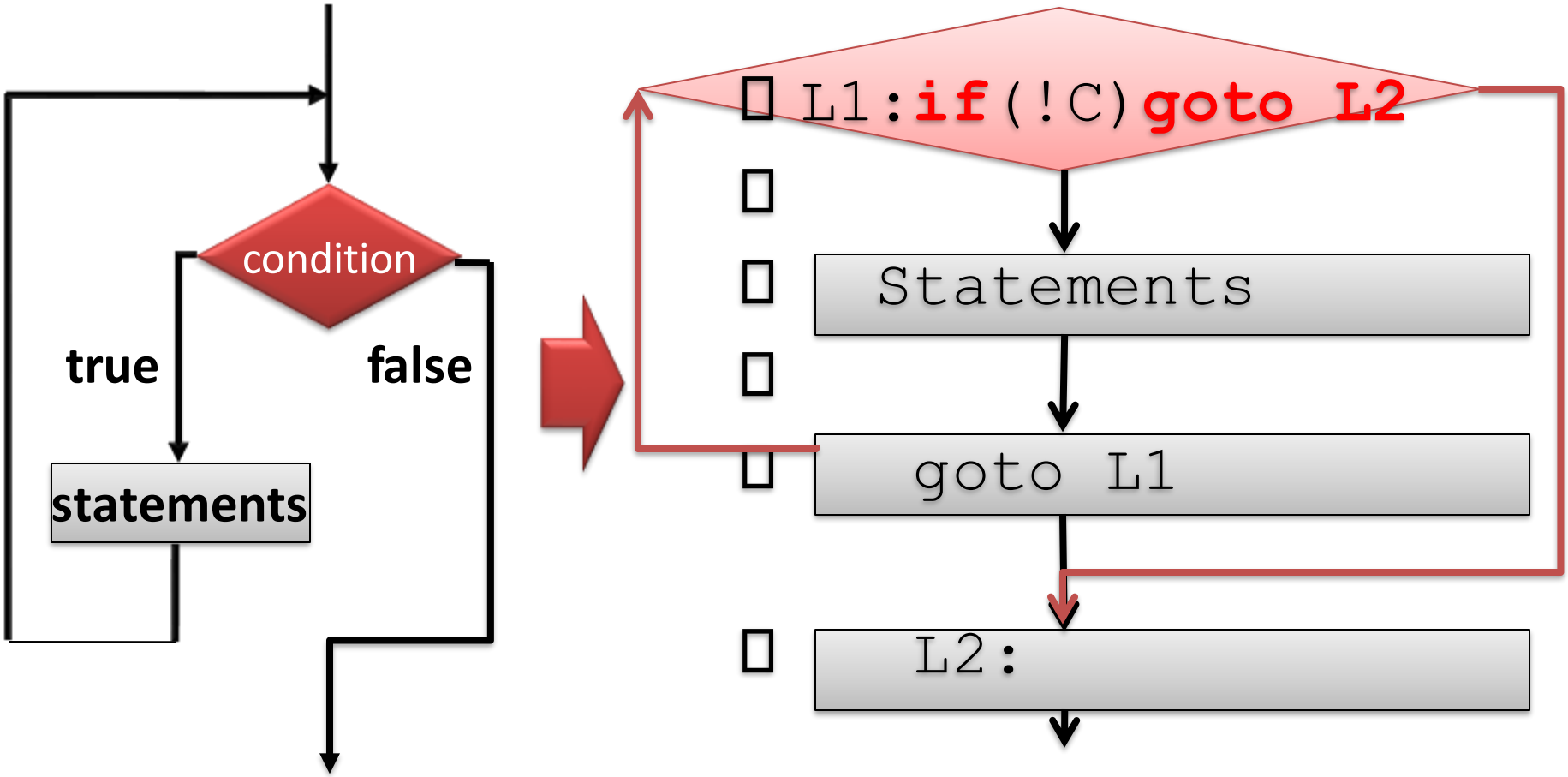
In Assembly language: No while loop

- Assembly language support for while loop
- While get implemented using **if** and **goto** statement
- goto statement uses **Label**
- **while** get converted to **if goto**

```
while(Cond){
    STMTS;
}
Next STMT;
```

```
L1:if(!Cond)goto L2;
    STMTS;
    goto L1;
L2:Next STMT
```

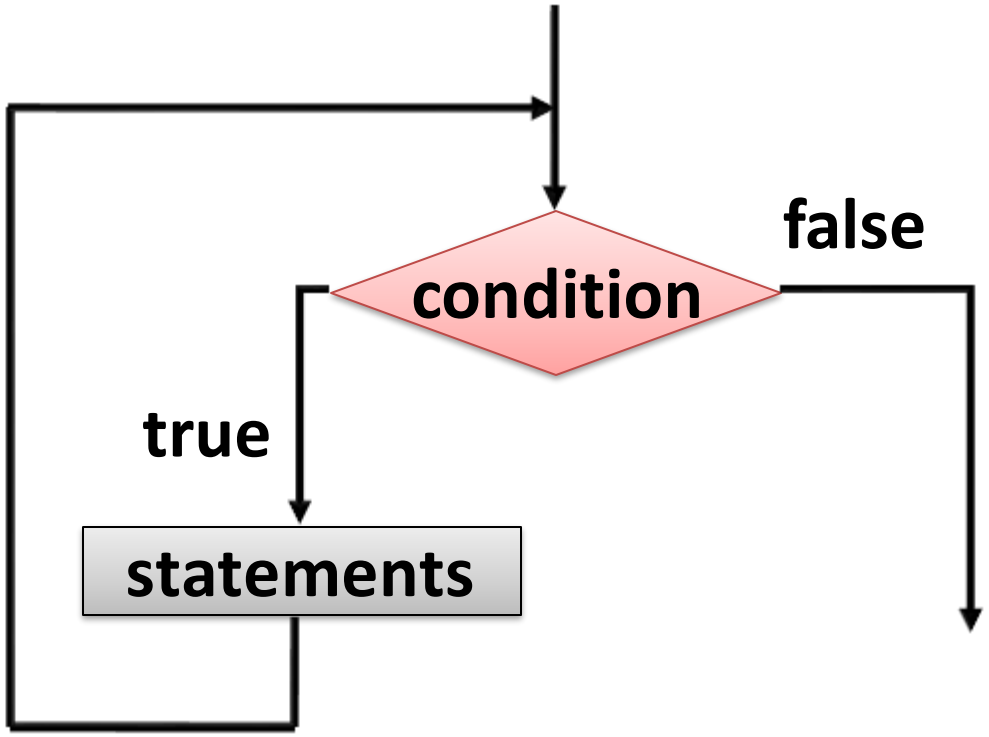
While statement using goto



While loop

```
while(condition)
     statement;

while(condition){
     statement1;
     statement2;
}
```




```
int i = 10;
while(i > 0) {
     printf("i=%d\n", i);
     i = i - 1;
}
```

Forever loops and never loops

- Always true condition - a loop that runs forever
- Always false - a loop never runs at all.

```
int count=0;
while(count !=0)
   printf("Hi .. \n");// never prints

while (count=1)//insidious error!!!
   count = 0;
```

How to count using while

1. First, outside the loop, initialize the counter variable
2. Test for the counter's value in the Boolean
3. Do the body of the loop
4. Last thing in the body should change the value of the counter!

```
1.      i = 1;
2.      while(i <= 10)   {
3.          printf("i=%d\n", i);
4.          i = i + 1;
        }
```

The do-while loop

- bottom-tested loop (posttest) or exit controlled loop
- One trip through loop is guaranteed, i.e. statement is executed at least once

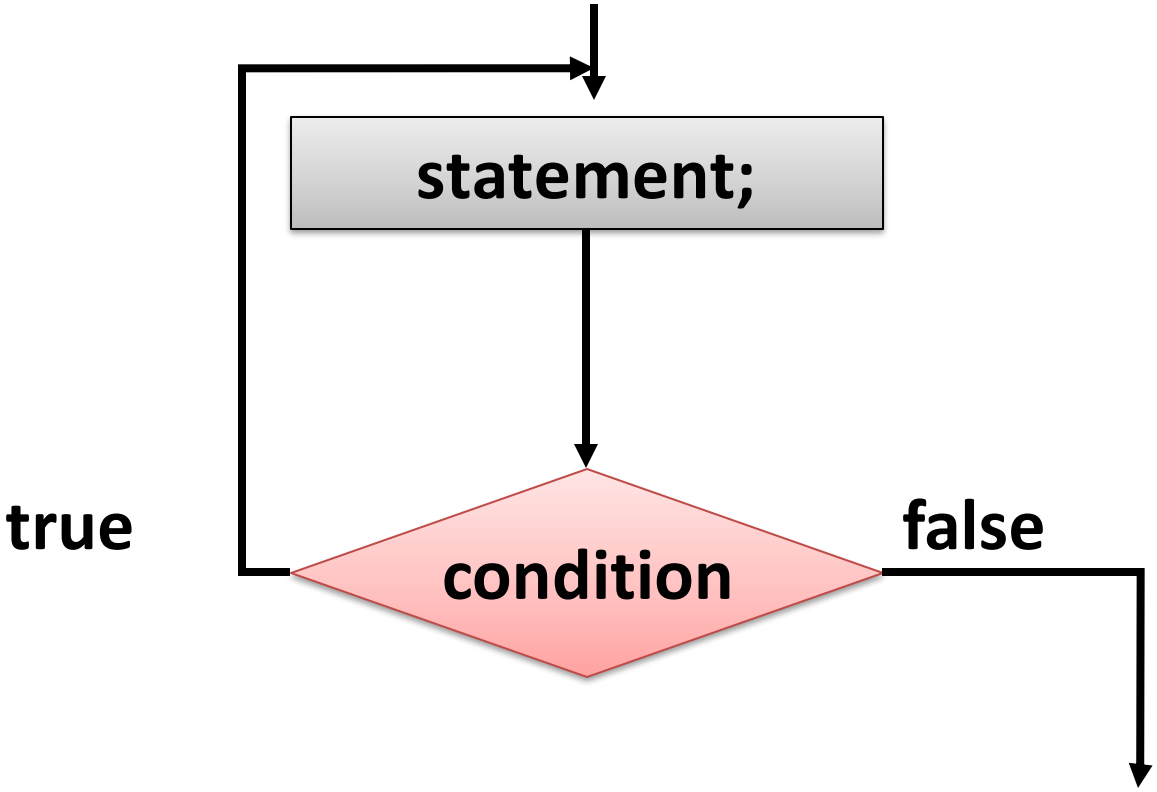
```
do
    statement
while (loop_condition);
```

```
do {
    statement1;
    statement2;
} while (loop_condition);
```

Usually!

do-while loop

```
do {
   statement;
}while(condition);
```



do while loop Examples

```
i = 0;
do {
   i++;
   printf("%d\n", i);
} while(i < 10);
```

```
 do {
    printf("Enter a value>0: ");
    scanf("%lf", &val);
} while(val <= 0);
```

Thanks